TESTING SQL 4

Q1. Category-wise Product Sales Ranking Analysis

Scenario

You are analyzing product sales data for a retail performance dashboard.
Management wants to identify the best-selling products within each category and compare their sales performance against other products in the same category.
The business wants a report showing
total quantity sold for **each product** and its **rank** within the category based on **sales volume.**

Concepts Tested

- INNER JOIN
- Window Function (RANK)
- Aggregation
- GROUP BY
- Filters
- ORDER BY / Sorting

Problem Statement

Write a SQL query to generate a category-wise product sales report.
Requirements:

- Calculate the total quantity sold for each product.
- Join the Categories, Products, and Order_Items tables.
- Rank products within each category based on total quantity sold in descending order.
- Only include products whose total quantity sold is greater than 5.
- Display category name, product name, total quantity sold, and sales rank.
- Sort the final output by category name and sales rank.

Required Columns

Column Namecategory_name product_name total_quantity_soldsales_rank

Database Schema

Table: Categories

category_id	category_name
1	Electronics
2	Furniture

Table: Products

product_id	name	category_id
1	Laptop	1
2	Mobile	1
3	Desk	2
4	Chair	2

Table: Order_Items

order_id	product_id	quantity
101	1	5
102	2	10
103	3	7
104	4	7
105	3	4

Expected Output

category_name	product_name	total_quantity_sold	sales_rank
Electronics	Mobile	10	1
Furniture	Desk	11	1
Furniture	Chair	7	2

📌 Constraints

- Use joins between all required tables.
- Use RANK() window function.
- Use SUM() aggregation.
- Include only products with total sales greater than 5.
- Products without orders should not appear.
- Sort by category_name and sales_rank.

Testing Sql Question 6

Database Description

The customer_orders database stores customer details and their order transactions. It helps the sales team analyze purchasing activity and identify customers based on their total spending.

Schema Details

The database contains the following two tables:

- Customers
- Orders

Sample Data

The following sample data is available in the Customers and Orders tables:

- Orders

Customers Table

Column Name	Data Type	Description
customer_id	INT (PK)	Unique identifier for customers
customer_name	VARCHAR(100)	Name of the customer

Orders Table

Column Name	Data Type	Description
order_id	INT (PK)	Unique order ID
customer_id	INT (FK)	Customer who placed the order
order_date	DATE	Date when the order was placed
order_amount	DECIMAL(10,2)	Total amount of the order

Sample Data

The following sample data is available in the Customers and Orders tables:

Customers Table

customer_id	customer_name
1	Alice
2	Bob

Orders Table

order_id	customer_id	order_date	order_amount
101	1	2024-03-01	100.00
102	1	2024-03-05	200.00
103	2	2024-03-02	300.00
104	2	2024-03-07	250.00

Problem Statement

The sales team wants to identify customers whose total spending across all their orders is greater than 400.
Use a subquery with an aggregate function to identify these customers. Then, display all orders placed by the identified customers.

Query to Be Implemented

Write an SQL query to return the following details:

- customer_name
- order_date
- order_amount

The query must meet the following requirements:

- Use the Customers and Orders tables.
- Join the two tables using customer_id.

- Use one subquery.
- Use the SUM() aggregate function to calculate each customer’s total spending.
- Use WHERE IN to identify customers whose total order amount is greater than 400.
- Display all orders placed by the identified customers.

Expected Output

Expected Output

Expected Output

customer_name	order_date	order_amount
Bob	2024-03-02	300.00
Bob	2024-03-07	250.00